KAKEIBO APP

セキュリティチェック報告書

生成元: docs/security-check.md | 生成日時: 2026/04/24 23:37

セキュリティチェック報告書

1. 実施概要

- 対象アプリ: かわいい家計簿 / `kakeibo-app`
- 本番 URL: `https://kawaii0214.vercel.app`
- 実施日: 2026-04-24（最終更新: 2026-04-24）
- 実施範囲: 依存パッケージ監査、ビルド検証、静的シークレット検査、API 防御状況の確認、本番 URL 確認

2. 数値サマリー

- セキュリティ総合評価: `93%`
- `npm audit` 脆弱性件数: `0`
- Critical: `0`
- High: `0`
- Moderate: `0`
- Low: `0`
- Git 管理下のハードコード秘密情報検出件数: `0`
- Git 管理下の `.env` 系ファイル件数: `1`
- 管理対象の `.env` 系ファイル: `.env.example` のみ
- `POST` API ルート数: `17`
- `same-origin` 防御あり: `16 / 17`（`94.1%`）
- `rateLimit` 適用あり: `7 / 17`（`41.2%`）
- ビルド失敗件数: `0`
- 非ブロッキング build warning 件数: `2`

3. 総合評価の算出方法

- 今回の総合評価は、静的検査とビルド検証の結果を 100 点満点で重み付けして算出
- 依存脆弱性: `30 / 30`
- シークレット管理: `25 / 25`
- リクエスト起点保護: `18.8 / 20`
- レート制限: `4.1 / 10`
- 品質ゲート: `15 / 15`
- 合計: `92.9 / 100` → 四捨五入して `93%`

3.1 配点の考え方

- 依存脆弱性は `npm audit` の結果をそのまま反映し、`0 件` のため満点
- シークレット管理は Git 管理下のハードコード秘密情報 `0 件` を満点評価
- リクエスト起点保護は `same-origin` を基本指標とし、`16 / 17` のため `94.1%` を 20 点へ按分
- レート制限は `7 / 17` の適用率 `41.2%` を 10 点へ按分
- 品質ゲートは ESLint、TypeScript、`next build` が通過しているため満点

3.2 解釈

- `93%` は「現時点の静的・構成・ビルド観点では高水準だが、追加改善余地あり」という評価
- 主な改善余地は `rateLimit` の適用拡大と、外部からの動的ペネトレーションテスト未実施の 2 点

4. 実施コマンド

- `npm audit --json`
- `./node_modules/.bin/eslint.cmd .`
- `node ./node_modules/typescript/bin/tsc --noEmit --pretty false`
- `npm run build`
- `git grep -n -I -E "(AIza ... |sk_live_ ... |PRIVATE KEY ... |AKIA ... |ghp_ ... )" -- .`
- `git ls-files .env .env.local .env.example .env.preview.local .vercel`
- API ルート走査:

```
POST_ROUTES=17
SAME_ORIGIN=16
RATE_LIMITED=7
MISSING_SAME_ORIGIN:
src/app/api/auth/send-email/route.ts
```

5. 結果詳細

4.1 依存パッケージ監査

- `resend` SDK 依存を外し、Resend REST API への直接送信に切り替えた後で `npm audit` を再実行
- その結果、依存関係由来の既知脆弱性は `0 件`
- 現時点での依存数は `582` で、そのうち production 依存は `176`

4.2 シークレット管理

- Git 管理下ファイルに対する静的スキャンでは、典型的な API キーや秘密鍵のハードコードは検出されず
- `.env` 、`.env.local` 、`.vercel` は Git 管理対象外
- Git 管理下に残っている環境変数ファイルは `.env.example` のみ

4.3 API 防御

- `POST` を持つ API ルートは `17`
- そのうち `16` ルートに `requireSameOrigin()` が入っており、一般的なブラウザ起点の CSRF 対策を実施
- `7` ルートに `rateLimit()` が入り、短時間連打の抑制を実施
- `src/app/api/auth/send-email/route.ts` は `same-origin` の代わりに Supabase hook の JWT / 署名検証で保護

4.4 品質ゲート

- ESLint: `0` errors / `0` warnings
- TypeScript: `0` errors
- `next build` : 成功
- ビルド警告は Auth0 依存からの warning が `2` 件あるが、出荷ブロッカーではない

6. 残課題と所見

- `rateLimit` は全 `POST` API に一律ではないため、将来的には対象拡大の余地あり
- Auth0 の `dpopUtils.js` に起因する build warning は依存側の挙動で、現状は機能阻害なし
- 今回は静的検査とビルド検証が中心であり、外部からの動的ペネトレーションテストまでは未実施
- LINE / Google OAuth の Callback URL は本番 URL に合わせて各開発者コンソールへ登録が必要

7. 総合判定

- 今回の実施範囲では、リポジトリ内の依存脆弱性は `0 件`
- 重大な秘密情報の Git 混入も検出されず
- ビルド、型検査、Lint を通過しており、現時点のコードベースはデプロイ可能な状態
- 総合評価は `93%` で、本番運用に耐える状態だが、今後は rate limit の拡張でさらに引き上げ可能